ANKARA YILDIRIM BEYAZIT UNIVERSITY

FACULTY OF AERONAUTICS AND ASTRONAUTICS

DEPARTMENT OF AEROSPACE ENGINEERING

PROJECT

AEE 332 - PROPULSION SYSTEMS I

GROUP 11

Kazım Buğra SITKI 21110111058

Mustafa ÜÇÖZ 21110111006

Oğuzhan GÜLER 21110111036

Sinan Emre ÖZEN 21110111052

Mehmet AKIN 21110111012

İbrahim Hakkı KOÇ 22110111023

May 3, 2025

ABSTRACT

In this project, we perform a parametric cycle analysis of an ideal turbojet engine with an afterburner. Using a MATLAB script we developed, we investigate how key performance parameters—such as specific thrust, specific fuel consumption, and fuel-to-air ratio—change across a range of flight Mach numbers and compressor pressure ratios. To assess the accuracy of our model, we compare its results with data obtained from the PARA (Parametric Analysis of Aero-engines) tool. This comparison helps validate our approach and provides insight into how well simplified cycle analysis can capture engine behavior under different operating conditions. Overall, the study offers a practical framework for understanding the performance trends of afterburning turbojet engines through both custom code and reference data.

Contents

1 Nomenclature **4**

2 Introduction **5**

2.1 Turbojet Engine 5

2.2 Afterburner and Its Impact on Turbojet Engines 6

2.3 What is the Parametric Cycle Analysis (PCA) 6

2.4 The performance parameters for Ideal Turbojet Engine 6

2.4.1 Specific Thrust 6

2.4.2 Specific Fuel Consumption (SFC): 7

2.4.3 Fuel-air Ratio: 7

2.4.4 Thermal Efficiency: 7

2.4.5 Propulsive Efficiency: 7

2.4.6 Overall Efficiency: 7

3 Methodology **8**

3.1 Assumptions 8

3.2 Turbojet Engine Configurations 8

3.3 Parametric Cycle Analysis Equations for an Ideal Turbojet Engine 9

3.4 MATLAB Code Implementation 11

3.5 MATLAB GUI Code Implementation 12

3.5.1 Interface Structure 12

3.5.2 Calculation Process 13

3.5.3 GUI Graphics 14

4 Results and Discussion **15**

4.1 Specific thrust vs π_c 15

4.2 Specific Fuel Consumption vs π_c 16

4.3 Fuel/Air ratio vs π_c 18

5 References **23**

6 Appendices **25**

6.1 MATLAB CODE 25

6.2 MATLAB GUI CODE 37

1 Nomenclature

- γ: Ratio of specific heats
- c_p: Specific heat at constant pressure (J/kg·K)
- R: Gas constant (J/kg·K)
- a_0: Speed of sound at ambient conditions (m/s)
- T_0: Ambient temperature (K)
- τ_r: Temperature ratio due to ram compression
- M_0: Flight Mach number
- τ_λ: Enthalpy ratio of burner
- T_{t4}: Turbine inlet stagnation temperature (K)
- $\tau_{\lambda AB}$: Stagnation temperature ratio across afterburner
- T_{t7}: Afterburner exit stagnation temperature (K)
- τ_c: Temperature ratio across compressor
- π_c: Compressor pressure ratio
- V_9: Nozzle exit velocity (m/s)
- $F/\dot{m}_0$: Specific Thrust (N*s/kg)
- $\dot{m}_0$: Mass flow rate (kg/s)
- g_c: Gravitational conversion constant (1 in SI units)
- h_{PR}: Heating value of fuel (J/kg)
- S: Specific fuel consumption (kg/N·s)
- f: Fuel-air ratio
- η_T: Thermal efficiency
- η_P: Propulsive efficiency
- η_O: Overall efficiency

2 Introduction

2.1 Turbojet Engine

A turbojet engine is a type of airbreathing jet engine that produces thrust by expelling high-speed exhaust gases. It is one of the earliest forms of jet propulsion systems and is commonly used in high-speed military aircraft and some early commercial jets.

The fundamental principle of a turbojet engine is to accelerate a mass of air through a sequence of thermodynamic processes, converting chemical energy (from fuel) into kinetic energy (thrust). This process is governed by the Brayton cycle, a thermodynamic cycle describing the operation of gas turbine engines [1].

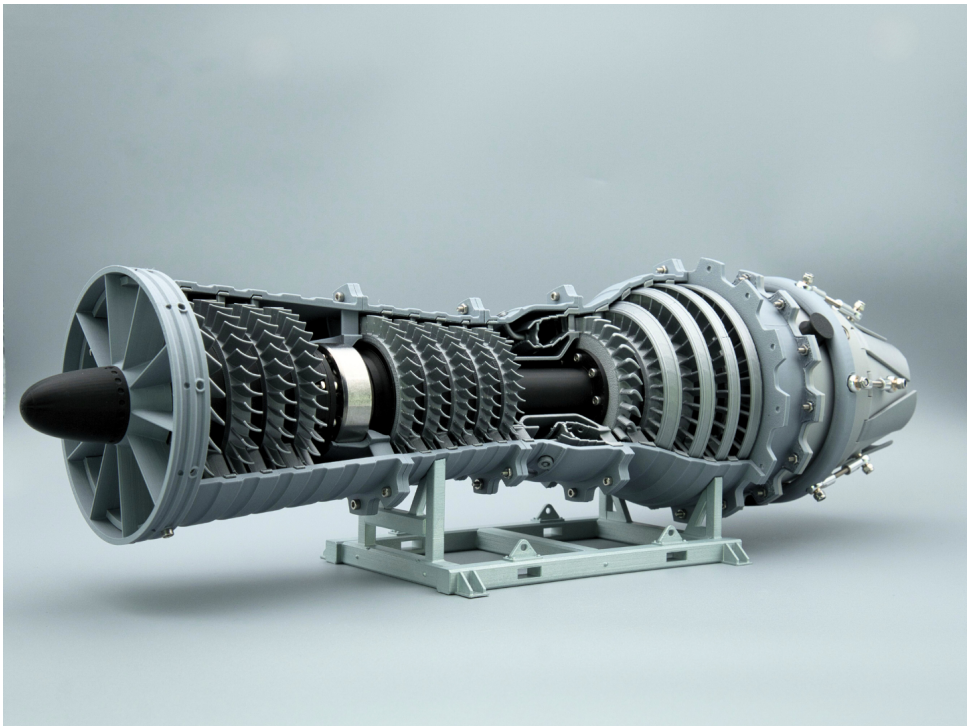

Figure 1: A 3D model of Turbojet engine

2.2 Afterburner and Its Impact on Turbojet Engines

An afterburner, also known as a reheat system, is an auxiliary combustion device installed downstream of the turbine in a turbojet or low-bypass turbofan engine [2, p. 250]. It is primarily used in military aircraft to provide a significant temporary increase in thrust, particularly during takeoff, supersonic flight, or combat maneuvers [3, p. 453].

In a conventional turbojet engine, only part of the energy from the combustion process is converted into mechanical work by the turbine, with the remaining energy expelled as high-temperature exhaust gases. These gases still contain residual oxygen, making it possible to inject additional fuel into the exhaust stream and ignite it within the afterburner [2, p. 254].

This secondary combustion process substantially increases the exhaust gas temperature and volume, leading to a greater acceleration of the flow through the nozzle, and thus producing a dramatic rise in thrust [3, p. 455].

2.3 What is the Parametric Cycle Analysis (PCA)

Parametric Cycle Analysis (PCA) is a method used to evaluate and predict the performance of air-breathing propulsion systems—such as turbojet, turbofan, or ramjet engines—by systematically varying key design and operating parameters.

Rather than focusing on a single operating condition, PCA explores how changes in parameters like compressor pressure ratio, turbine inlet temperature, flight Mach number, and fuel-to-air ratio affect engine performance. It provides critical insights for both preliminary design and optimization of engine configurations [4, p. 121].

2.4 The performance parameters for Ideal Turbojet Engine

2.4.1 Specific Thrust

Specific thrust is the ratio of the engine's thrust to the mass flow rate of incoming air. It serves as a measure of engine performance per unit airflow, indicating how effectively the engine produces thrust under varying operating conditions [12, 14].

2.4.2 Specific Fuel Consumption (SFC):

Specific Fuel Consumption is defined as the mass of fuel consumed per unit time per unit thrust produced. It reflects the fuel efficiency of the engine, with lower SFC values indicating more efficient propulsion systems [12, 14].

2.4.3 Fuel-air Ratio:

The fuel-air ratio denotes the proportion of fuel mass to air mass in the combustion process. It plays a crucial role in engine thermal regulation and combustion efficiency. Excessive fuel leads to overheating, while insufficient fuel can cause incomplete combustion [12, 14].

2.4.4 Thermal Efficiency:

Thermal efficiency quantifies the fraction of chemical energy in fuel that is converted into kinetic energy. It depends largely on the temperature rise across the combustion chamber and is directly influenced by pressure ratio and turbine inlet temperature [12, 14].

2.4.5 Propulsive Efficiency:

Propulsive efficiency evaluates how well the jet engine converts exhaust kinetic energy into useful thrust. Higher propulsive efficiency is achieved when the exhaust velocity closely matches the aircraft's flight speed [12, 14].

2.4.6 Overall Efficiency:

Overall efficiency is the product of thermal and propulsive efficiencies, providing a holistic measure of engine performance from energy conversion to thrust output [12, 14].

3 Methodology

3.1 Assumptions

1. All compression and expansion stages (diffuser, compressor, fan, turbine, and nozzle) are assumed to be reversible and adiabatic [3, p. 27].

2. Heat addition in the combustor is modeled as a constant-pressure process; the fuel mass flow is negligible relative to the air mass flow, so the total mass flow remains essentially unchanged [2, p. 13].

3. The working fluid is taken as air, behaving as a perfect gas with constant specific heats [5, p. 5].

4. The exhaust nozzles expand the gas back to ambient pressure [3, p. 458].

5. There are no bleed or cooling flows in the system [4, p. 130].

3.2 Turbojet Engine Configurations

At point 0, the ambient air moves from the free stream to the leading edge of the flight inlet at point 1. The air reaches the compressor face from the engine inlet at point 2 [6].

The compressor then increases both the pressure and temperature of the air. This process requires work input, and the resulting temperature increase depends on the efficiency of the compressor [4, 10]. The detailed thermodynamic treatment of axial and centrifugal compressor performance is essential for predicting this behavior [10]. The air leaves the compressor at point 3 and enters a diffuser that directs it into the combustion chamber. Fuel is injected and burned in the combustion chamber, increasing the gas temperature at point 4 [5, 11].

The hot, high-pressure gas then expands through the turbine, where energy is extracted to provide shaft power. As a result, both the pressure and temperature are reduced. This shaft power is used to drive the compressor and any auxiliary systems. After leaving the turbine at point 5, the gas is usually still at least twice the pressure of ambient due to the high inlet temperature [6, 4, 10]. A thorough understanding of turbine work and blade efficiency is critical at this stage [10].

The gas stream is then passed through point 6 to point 9, where it diffuses. A small pressure loss also occurs here [5].

An afterburner may be used at point 7. This is a secondary combustion chamber located just downstream of the turbine and upstream of the nozzle. It burns additional fuel through special injectors to increase thrust [4, 11]. The use of afterburners significantly alters engine performance and is covered in detail in turbojet augmentation methods [10].

A thruster nozzle is a converging duct that accelerates the exhaust gases to produce a high-speed jet and generates thrust [7]. For optimal performance, nozzle geometry is carefully matched to flight conditions [9].

For high Mach number flights, afterburners are often used to produce more thrust with the same engine structure. Turbojet engines are relatively inefficient at low Mach numbers, but are quite effective at supersonic speeds and in military applications [6, 11, 10].

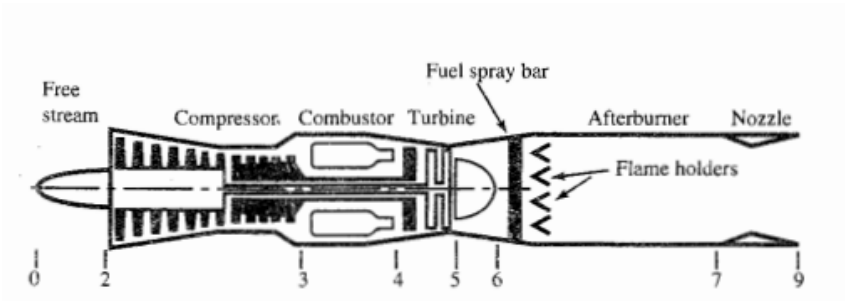

Figure 2: Turbojet Engine Configurations [6, p. 266]

3.3 Parametric Cycle Analysis Equations for an Ideal Turbojet Engine

Parametric cycle analysis provides a valuable framework for evaluating the thermodynamic performance of an ideal turbojet engine as a function of flight conditions and component efficiencies. The analysis begins by applying the conservation of mass and energy across each engine station, under the assumption of steady, one-dimensional, compressible, and inviscid flow. Key parameters such as flight Mach number M_0, altitude, compressor pressure ratio π_c, turbine inlet temperature T_{t4}, and afterburner exit temperature T_{t7} are used to evaluate the behavior of the system under idealized conditions [4, 10].

At station 0, total (stagnation) conditions are defined using isentropic relations that

relate static and stagnation properties through the freestream Mach number:

$$T_{t0} = T_0 \left(1 + \frac{\gamma - 1}{2} M_0^2\right), \quad P_{t0} = P_0 \left(1 + \frac{\gamma - 1}{2} M_0^2\right)^{\frac{\gamma}{\gamma-1}} \tag{1}$$

As the air passes through the diffuser and compressor, the stagnation pressure and temperature rise. The compressor temperature rise is given by:

$$T_{t3} = T_{t2} \left[1 + \frac{1}{\eta_c} \left(\pi_c^{\frac{\gamma-1}{\gamma}} - 1\right)\right] \tag{2}$$

The work input to the compressor is directly related to the pressure ratio and isentropic efficiency [5]. The fuel-to-air ratio f in the main combustor can be determined by applying the first law of thermodynamics:

$$f = \frac{c_p T_0}{h_{PR}} (\tau_\lambda - \tau_r \tau_c) \tag{3}$$

Following combustion, the high-temperature, high-pressure gas expands through the turbine, supplying the necessary work for the compressor. In afterburning configurations, additional fuel is injected downstream of the turbine, raising the temperature to T_{t7} before the flow enters the nozzle.

The thrust-specific fuel consumption (TSFC), specific thrust $F/\dot{m}_0$, and overall thermal efficiency η_o are derived from the momentum and energy balance across the control volume [6, 11]:

$$a_0 = \sqrt{\gamma R T_0} \tag{4}$$

$$\tau_r = 1 + \frac{\gamma - 1}{2} M_0^2 \tag{5}$$

$$\tau_\lambda = \frac{T_{t4}}{T_0} \tag{6}$$

$$\tau_{\lambda AB} = \frac{T_{t7}}{T_0} \tag{7}$$

$$\tau_c = (\pi_c)^{(\gamma-1)/\gamma} \tag{8}$$

$$\frac{V_9}{a_0} = \sqrt{\frac{2}{\gamma - 1} \left(\tau_\lambda - \tau_r \tau_c - \frac{\tau_\lambda}{\tau_r \tau_c}\right)} \tag{9}$$

$$\frac{F}{\dot{m}_0} = \frac{a_0}{g_c} \left(\frac{V_9}{a_0} - M_0\right) \tag{10}$$

$$S = \frac{f}{F/\dot{m}_0} \tag{11}$$

$$\eta_T = \frac{(\gamma - 1)\, c_p\, T_0 \left(\left(\frac{V_9}{a_0}\right)^2 - M_0^2 \right)}{2 f_{tot} h_{PR}} \tag{11}$$

$$\eta_P = \frac{2M_0}{(V_9/a_0 + M_0)} \tag{12}$$

$$\eta_O = \eta_T \eta_P \tag{13}$$

This parametric formulation enables rapid evaluation of engine performance over a range of flight conditions and design choices, supporting the conceptual design and optimization of modern turbojet systems [10, 13].

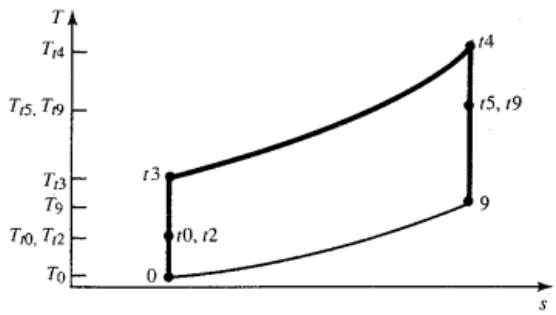

Figure 3: The T-s diagram of an ideal turbojet engine. [6]

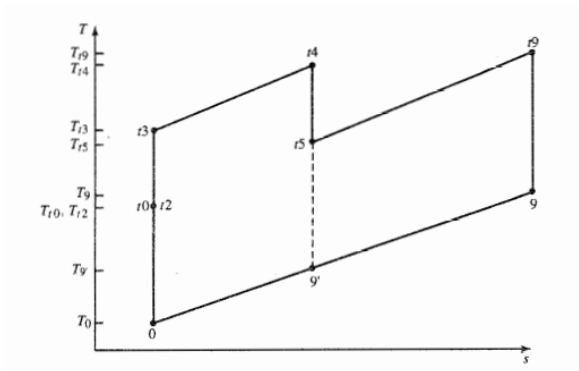

Figure 4: The T-s diagram for an ideal afterburning turbojet engine [6].

3.4 MATLAB Code Implementation

The MATLAB code begins by obtaining user-defined input parameters. These inputs include the range of Mach numbers, the range of compressor pressure ratios, the flight altitude (in feet), the total turbine inlet temperature (T_{t4}), and the total afterburner exit temperature (T_{t7}).

Based on the thermodynamic relationships presented in Section 3.1, the code implements a series of calculations to model the performance of an ideal turbojet engine with afterburner. Using these equations, the relevant performance parameters are computed over the specified operating ranges [1, 2, 3].

The script also defines several constants that are necessary for the computations. These constants are:

- $\gamma = 1.4$ (specific heats ratio for air)
- $c_p = 1004.83$ J/kg·K (specific heat capacity at constant pressure)
- $h_{PR} = 42.8 \times 10^6$ J/kg (fuel heating value)
- $R = \left(\frac{\gamma-1}{\gamma}\right) c_p$ (universal gas constant)
- $g_c = 1$ (proportionality constant; equals 1 in SI units)

Finally, the computed results are visualized using graphical plots, which illustrate the variation of engine performance metrics under different flight and design conditions. These plots serve to analyze how parameters such as Mach number, pressure ratio, and temperature affect specific thrust, fuel consumption, and efficiencies [4, 5].

3.5 MATLAB GUI Code Implementation

In this project, a MATLAB GUI (Graphical User Interface) has been developed to perform performance analysis of an ideal afterburning turbojet engine. The interface computes engine performance based on user-specified input parameters and presents the results graphically [7, 8].

3.5.1 Interface Structure

The main interface consists of three main panels:

1. **Input Parameters:** The user determines the Mach number range, altitude (ft), total temperature values (T_{t4} and T_{t7}), compressor pressure ratio range (π_c), and afterburner status via this panel. Inputs are taken using components such as `uieditfield`, `uiswitch`, and `uibutton`.

   When the `"Reset"` button is pressed, the `resetFields()` function is executed and all input fields are returned to their default values. At the same time, the graphics area is cleared.

   When the `"Run"` button is pressed, the `calculate()` function is called. This function is explained in the next section [7].

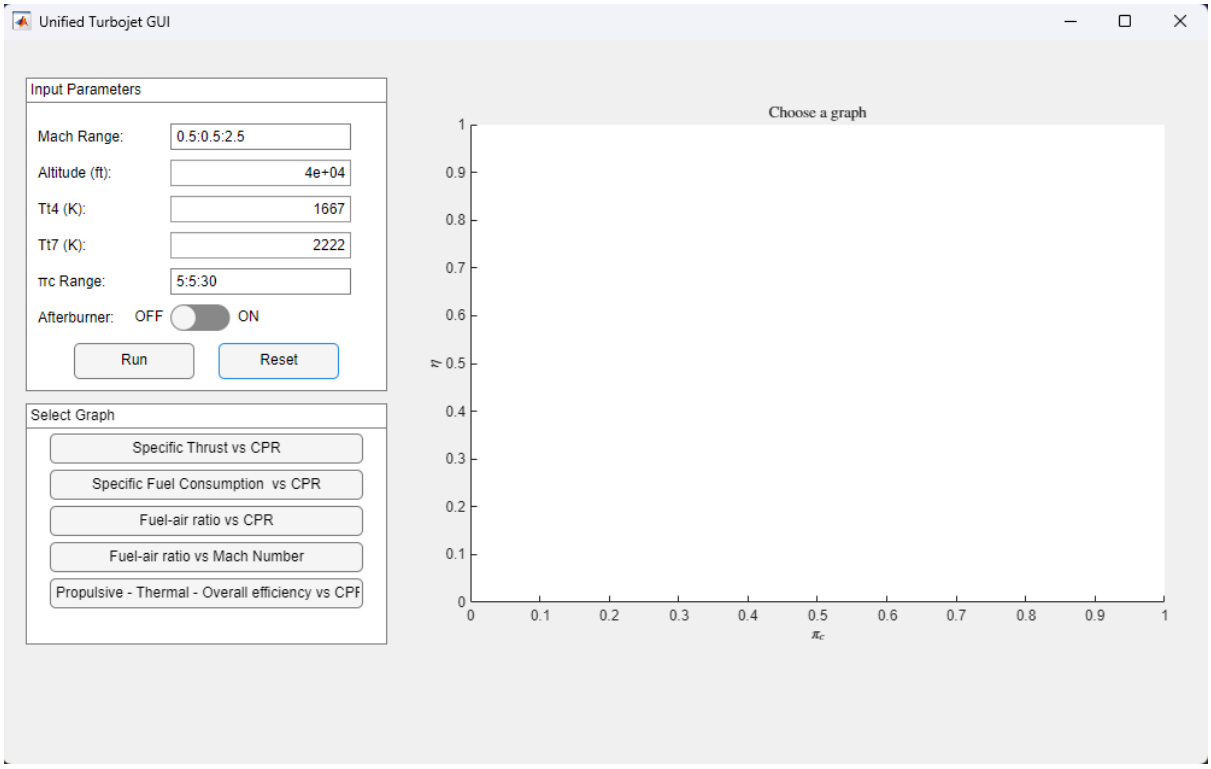

Figure 5: Graphical User Interface

2. **Graph Selection:** This panel allows the user to choose the desired graph to visualize the calculated data. Each button displays a graph of a different performance parameter (e.g., specific thrust vs. π_c, specific fuel consumption vs. π_c, efficiencies vs. π_c).

3. **Plot Area:** Using the `uiaxes` command, the appropriate data is plotted in this section depending on the selected graph type [8].

3.5.2 Calculation Process

When the `Run` button is pressed, the `calculate()` function is executed, performing the following steps [7]:

1. **Obtaining Input Parameters:** The Mach number and compressor pressure ratio ranges specified by the user are converted into vectors using the `eval` command. The altitude value is taken in feet and converted into meters.

2. **Reading Atmospheric Data:** The temperature and pressure values corresponding to the altitude are obtained from the `ATMTABLE.xlsx` file using linear interpolation with the `interp1` function.

3. **Basic Thermodynamic Calculations:** Thermodynamic parameters such as reaction rates (τ_r, τ_c, τ_λ, τ_t) and flight speed (a_0) are calculated. Depending on the afterburner status, thrust, specific fuel consumption (SFC), fuel-air ratio (f), and efficiencies (propulsive, thermal, total) are computed.

4. **Storing Data:** All computed performance values are stored in a global structure named `results`, which is then used for plotting.

3.5.3 GUI Graphics

The user can visually examine the results by selecting the desired graph type. Graphs are generated using the `plotGraph(type)` function. Each graph compares the selected performance metric across different Mach numbers or π_c values [8].

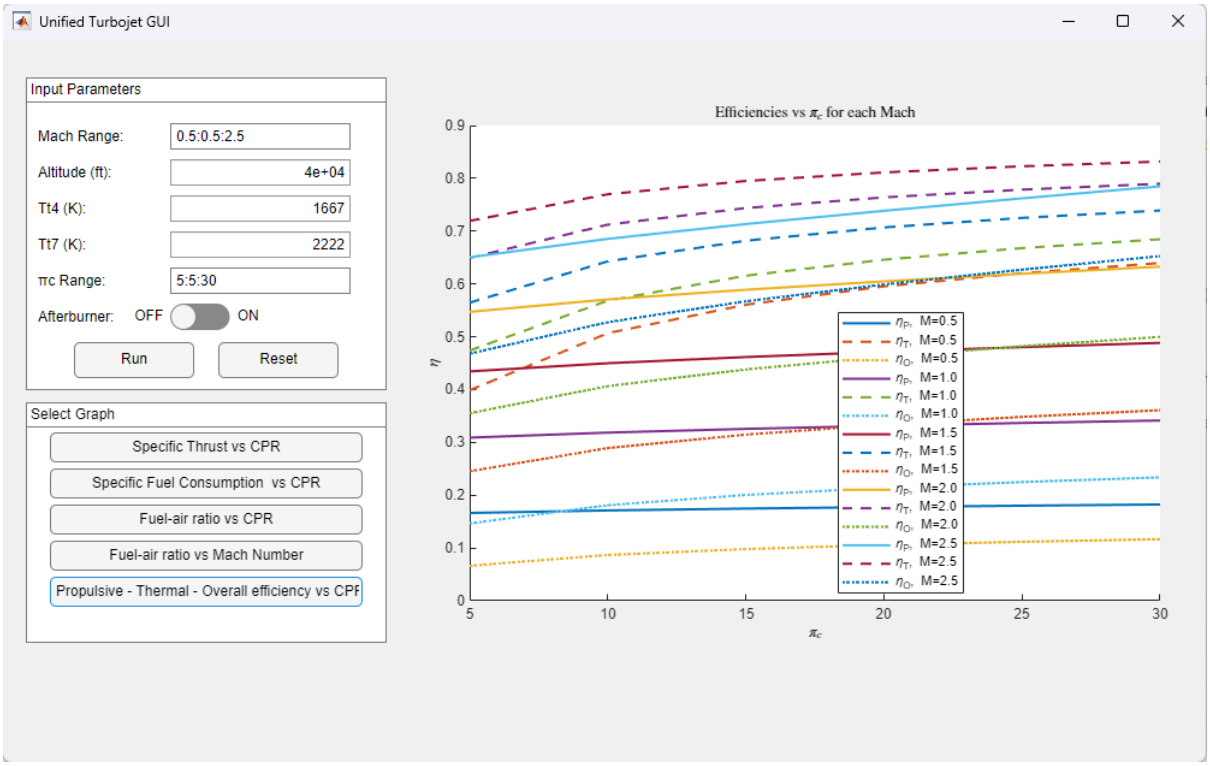

Figure 6: Graphical User Interface

4 Results and Discussion

4.1 Specific thrust vs π_c

Impact of Rising Mach Number on Specific Thrust

As the flight Mach number escalates, specific thrust experiences a substantial rise, primarily driven by the afterburner's contribution. The key factor is that at elevated Mach numbers, the incoming airflow possesses greater kinetic energy, allowing the turbine-combustion system to harness this energy more efficiently. The afterburner further amplifies thrust by elevating exhaust gas temperature, thereby increasing exhaust velocity. Consequently, specific thrust demonstrates a clear upward trend between Mach 0.5 and 2.5, though the rate of improvement varies with the compressor pressure ratio (π_c).

Role of Compressor Pressure Ratio (π_c)

Increasing the compressor pressure ratio enhances specific thrust—but only up to a limit. At lower π_c values, raising the pressure ratio improves compression efficiency, leading to higher turbine exit temperatures. When coupled with afterburner operation, this results in greater exhaust velocity and thrust. However, beyond $\pi_c \approx 20$, the gains in specific thrust begin to diminish. This is due to thermal constraints: excessive compression pushes turbine temperatures toward material limits, restricting further performance improvements.

Interaction of Mach Number and π_c

The highest specific thrust values occur at high Mach numbers ($M > 2.0$) combined with moderate-to-high π_c (15–25). In contrast, at subsonic speeds ($M < 1.0$), even extremely high π_c values yield marginal thrust improvements because the incoming air's kinetic energy is insufficient for the afterburner to make a substantial difference.

Afterburner's Influence

While the afterburner significantly boosts specific thrust across all Mach regimes, it comes at the cost of dramatically higher fuel consumption, adversely affecting thermal efficiency.

Since this analysis focuses on ideal performance, fuel efficiency trade-offs are not explored in depth.

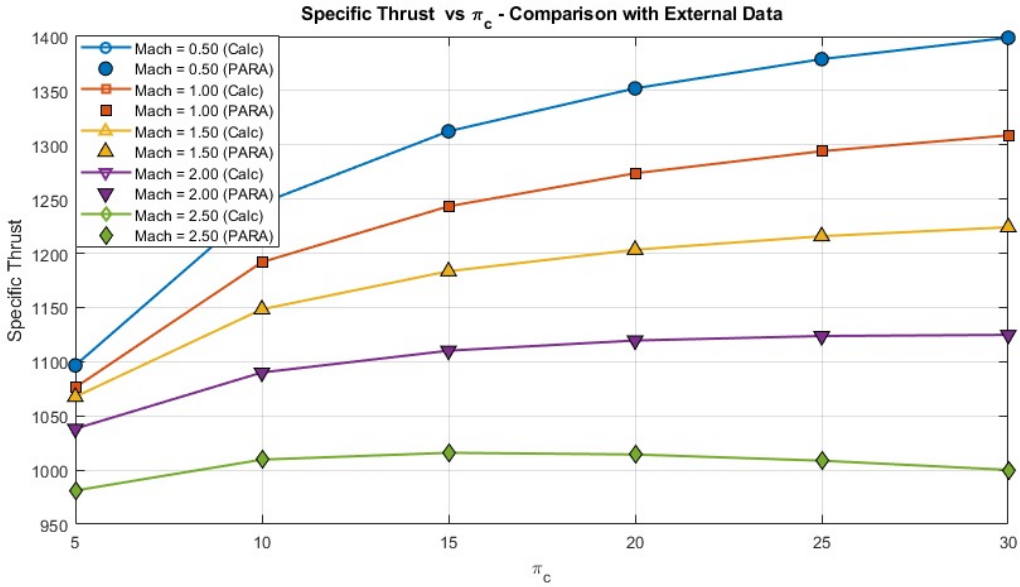

Figure 7: Specific Thrust vs π_c - Comparison with External Data

4.2 Specific Fuel Consumption vs π_c

Effect of Increasing Mach Number

As flight Mach number increases, specific fuel consumption (SFC) typically exhibits a decreasing trend. This improvement occurs because the engine generates greater thrust per unit fuel flow at higher Mach numbers due to enhanced dynamic pressure and improved propulsive efficiency. However, this beneficial effect becomes less pronounced when operating with an afterburner, as the afterburner's substantial fuel flow increment dominates the SFC characteristics regardless of flight speed. While SFC shows modest reduction with Mach number at lower compressor pressure ratios (π_c), this advantage attenuates significantly at higher pressure ratio conditions.

Effect of Compressor Pressure Ratio (π_c)

Elevating the compressor pressure ratio generally reduces SFC, particularly at low-to-medium Mach number regimes. The higher pressure ratio improves core thermal efficiency, enabling more effective energy extraction from each unit of fuel. The afterburner's presence, however, mitigates this improvement because it introduces disproportionate fuel

flow increases without corresponding efficiency gains. Beyond a critical π_c threshold (approximately 20–25), the SFC reduction becomes progressively marginal, demonstrating clear saturation behavior.

Combined Effects and Operational Trade-offs

While the combination of high π_c and elevated Mach number delivers both superior thrust output and relatively favorable SFC characteristics, the afterburner's inherent compromise remains evident. Although providing substantial thrust augmentation, this comes at the expense of significantly degraded fuel efficiency. SFC values in afterburner operation markedly exceed those of dry (non-afterburning) operation, especially in low Mach number regimes where ram pressure contributions are limited.

Afterburner Impact on Efficiency

The afterburner represents the primary driver of increased SFC, particularly at low Mach numbers and low π_c conditions. In these operational regimes, the thrust benefit from afterburner operation proves relatively small compared to its substantial fuel flow penalty, resulting in poor fuel efficiency. While operationally valuable for short-duration high-thrust requirements, sustained afterburner use leads to prohibitively high fuel consumption rates.

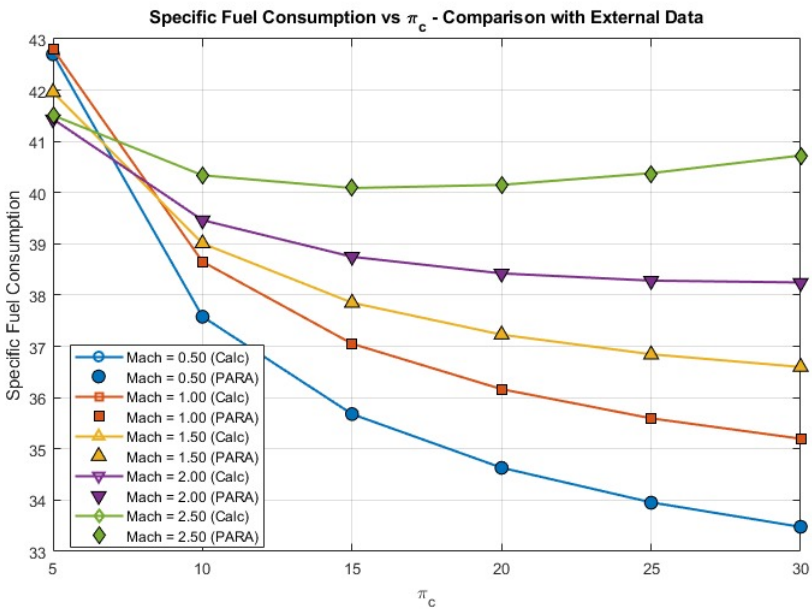

Figure 8: Specific Fuel Consumption vs π_c - Comparison with External Data

4.3 Fuel/Air ratio vs π_c

Effect of Mach Number

As the Mach number increases, the required fuel/air ratio tends to decrease slightly. This is because at higher speeds, the engine receives more air, and combustion becomes more efficient. Even though the afterburner adds extra fuel, the overall fuel/air ratio goes down a bit since more air is entering the engine at high Mach numbers.

Effect of Compressor Pressure Ratio (π_c)

When the pressure ratio increases, the fuel/air ratio usually increases. A higher pressure ratio means that the air is more compressed before combustion, which allows more fuel to be burned. This results in a higher temperature after combustion, especially when combined with afterburning. However, if the pressure ratio becomes too high, the increase in fuel/air ratio becomes smaller due to limitations such as turbine temperature limits.

Effect of Afterburner

The use of an afterburner clearly increases the fuel/air ratio. This is because the afterburner injects extra fuel into the exhaust to increase temperature and thrust. However, this extra fuel is not burned in the main combustor and does not improve core thermal efficiency. Therefore, fuel/air ratio values with afterburner are much higher compared to engines without afterburning.

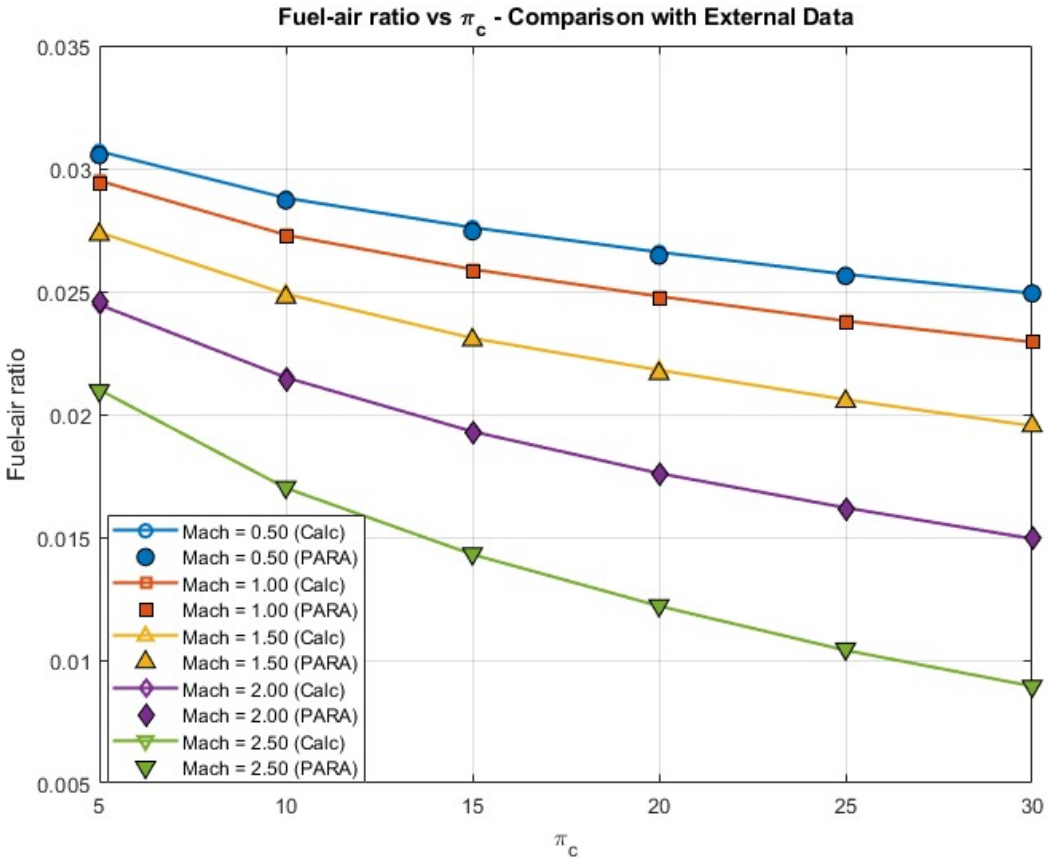

Figure 9: Fuel-air ratio vs π_c - Comparison with External Data

Effect of Mach Number and π_c on Fuel/Air Ratio

As Mach number increases, the fuel/air ratio generally decreases for all values of compressor pressure ratio. This is mainly because higher Mach numbers bring more air into the engine, reducing the amount of fuel needed per unit of air. Engines with higher π_c values tend to operate with lower fuel/air ratios, due to better compression and combustion efficiency. The afterburner increases fuel use overall, but the decreasing trend with Mach number remains consistent.

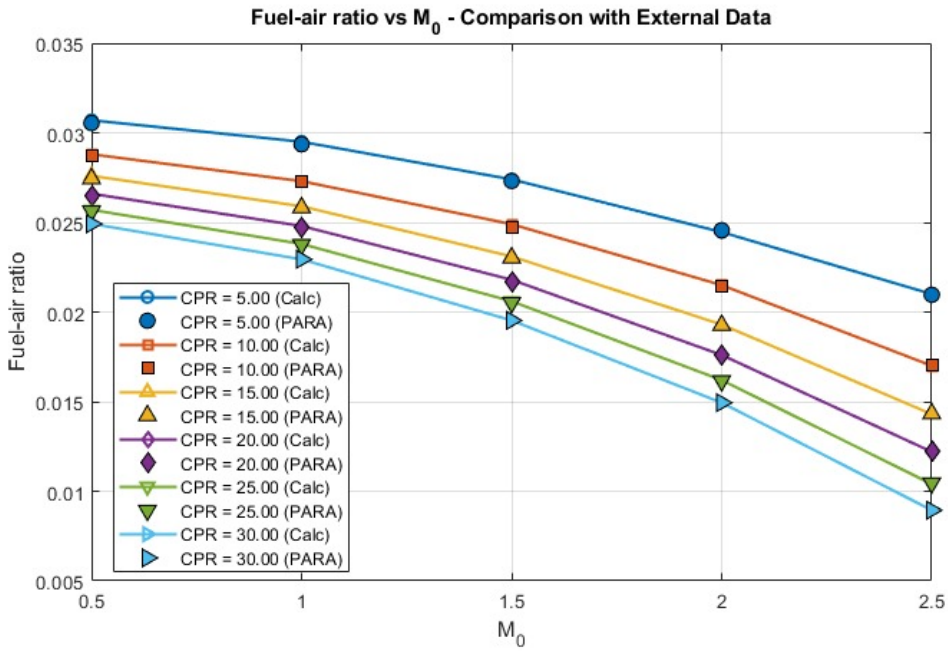

Figure 10: Fuel-air ratio vs M_0 - Comparison with External Data

Propulsive Efficiency

When π_c increases, propulsive efficiency usually goes down, especially at low Mach numbers. This is because the engine pushes out the exhaust gases faster, and if the exhaust speed is much higher than the aircraft speed, some energy is wasted. Propulsive cycle analysis (PCA) shows that the best propulsive efficiency happens when the exhaust speed is close to the aircraft's speed. With high π_c and afterburning, this balance is harder to keep. At higher Mach numbers, this loss becomes less important, but the general trend stays the same.

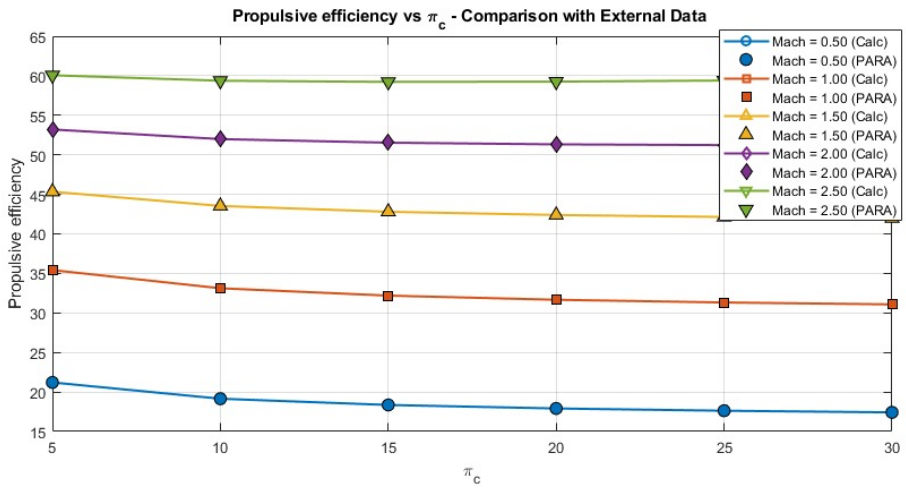

Figure 11: Propulsive efficiency vs π_c - Comparison with External Data

Thermal Efficiency

As the compressor pressure ratio increases, thermal efficiency also increases. This happens because the engine can get more useful energy from the fuel when the air is compressed more before combustion. In propulsive cycle analysis (PCA) results, we can clearly see this trend, especially up to around $\pi_c = 20$–25. However, in engines with an afterburner, extra fuel is added after the turbine. Since this fuel doesn't help power the turbine, it doesn't improve thermal efficiency. So, even though thermal efficiency rises with π_c, the afterburner makes the improvement look smaller.

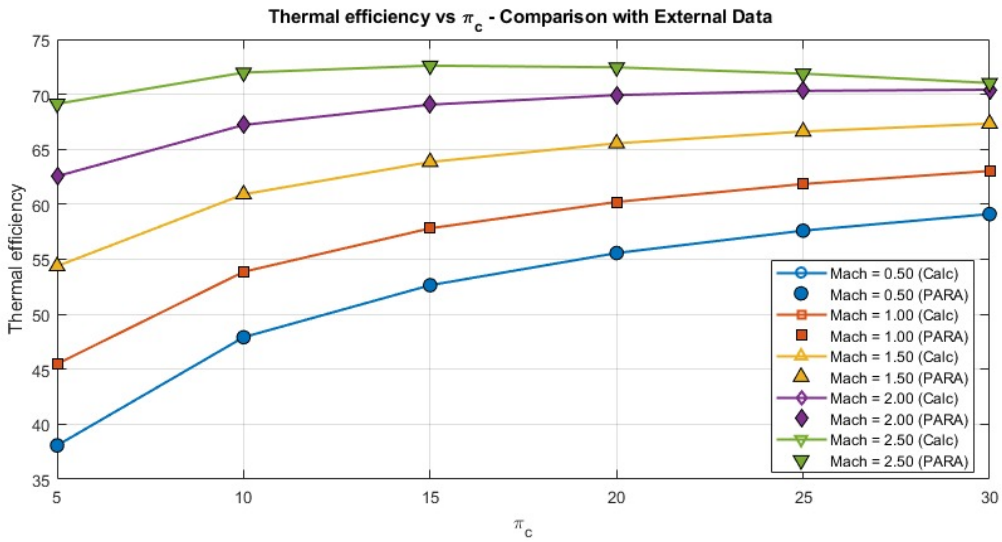

Figure 12: Propulsive efficiency vs π_c - Comparison with External Data

Overall Efficiency

Overall efficiency depends on both thermal and propulsive efficiency. As π_c increases, overall efficiency first goes up, because thermal efficiency improves a lot. But after a point (around $\pi_c = 15$–20), the drop in propulsive efficiency and the high fuel use from the afterburner reduce the benefits. So, overall efficiency reaches a peak at medium π_c values, and then either stays flat or goes down slightly at very high pressure ratios.

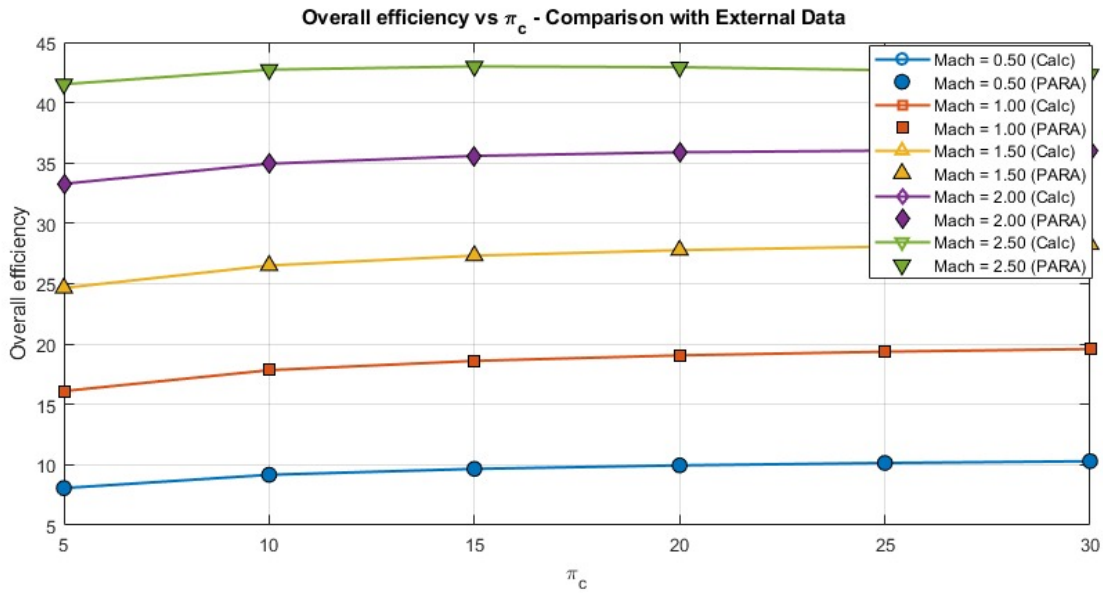

Figure 13: Overall efficiency vs π_c - Comparison with External Data

5 References

References

[1] Jones, S. M., *An Introduction to Thermodynamic Performance Analysis of Aircraft Gas Turbine Engine Cycles Using the Numerical Propulsion System Simulation Code*, NASA Technical Memorandum 2007-214690, NASA Glenn Research Center, Cleveland, OH, 2007, pp. 4.

[2] Mattingly, J. D., *Elements of Propulsion: Gas Turbines and Rockets*, AIAA Education Series, American Institute of Aeronautics and Astronautics, Reston, VA, 2005, pp. 250–255.

[3] Hill, P., and Peterson, C., *Mechanics and Thermodynamics of Propulsion*, 2nd ed., Addison-Wesley, Reading, MA, 1992, pp. 453–460.

[4] Mattingly, J. D., Heiser, W. H., and Pratt, D. T., *Aircraft Engine Design*, AIAA Education Series, American Institute of Aeronautics and Astronautics, Reston, VA, 1996, pp. 121–130.

[5] Anderson, J. D., *Fundamentals of Aerodynamics*, 5th ed., McGraw-Hill, New York, 2011, pp. 5–10.

[6] Mattingly, J.D., 2005, *Elements of Gas Turbine Propulsion*, AIAA Education Series, AIAA, Reston, VA.

[7] Kong, C., Kho, S., and Park, G., 2013, "Development of Practical Integral Condition Monitoring System for a Small Turbojet Engine Using MATLAB/SIMULINK and LabVIEW," *International Journal of Turbo Jet-Engines*, **31**(1), pp. 1–10. https://doi.org/10.1515/tjj-2013-0030

[8] Attaway, S., 2022, *MATLAB: A Practical Introduction to Programming and Problem Solving*, 6th ed., Butterworth-Heinemann, Oxford, UK, pp. 255–275.

[9] Curtis, J., and Hoke, J. L., 2021, "Design and Optimization of Supersonic Nozzles for Turbojet Propulsion," *Aerospace Science and Technology*, **115**, 106825. https://doi.org/10.1016/j.ast.2021.106825

[10] Saravanamuttoo, H. I. H., Rogers, G. F. C., and Cohen, H., *Gas Turbine Theory*, 6th ed., Pearson Education, Harlow, UK, 2009, pp. 63–70, 127–135, 179–185, 226–230.

[11] Hill, P., and Peterson, C., *Mechanics and Thermodynamics of Propulsion*, 2nd ed., Pearson, Upper Saddle River, NJ, 2012, pp. 453–460.

[12] Acharya, S., Sahoo, D., Parida, P., and Barik, B., 2022, "Thermal and Propulsive Performance Analysis of an Ideal Turbojet Engine," *IOSR Journal of Engineering*, Vol. 12, No. 1, pp. 1–7.

[13] Curti, L., Cerri, G., and Mancinelli, A., 2020, "Analytical Evaluation of Turbojet Performance for Conceptual Design Applications," *International Journal of Turbo Jet-Engines*, **37**(2), pp. 125–136. https://doi.org/10.1515/tjj-2020-0005

[14] Salari, M., Amirkhani, A., and Moghiman, M., 2018, "Exergy Analysis and Optimization of a Turbojet Engine Using Genetic Algorithm," *Energy*, 142, pp. 592–599. https://doi.org/10.1016/j.energy.2017.10.079

6 Appendices

6.1 MATLAB CODE

```
    clear; clc;
ab_status = lower(input("Afterburner is 'on' or 'off'? (on/off): ", 's'
    ));

if strcmp(ab_status, "on")
    disp("Afterburner is ON. PCA analysis is starting... ");

%inputs

M_0_start = input("mach start value:");
M_0_end = input("mach end value:");
altitude_ft = input("altitude:");
T_t4 = input("T_t4:");
T_t7 = input("T_t7:");
pi_c_start = input("pi_c start value:");
pi_c_end = input("pi_c end value:");

% Mach ve pi_c arrays
Mach_array = M_0_start : 0.5 : M_0_end;
pi_c_array = pi_c_start : 5 : pi_c_end;
gc=1;

% constants
gamma = 1.4;            % specific heats ratio
c_p = 1004.83;          % specific heat capaticity (J/kg.K)
h_PR = 42800e3;         % hpr (J/kg)
R = ((gamma-1)/gamma)*c_p ; % universal Gas constant (J/kg.K)
g_c = 1;    %its 1 since we work in SI units

%% temperature, altitude and pressure

% feet to meter
altitude_m = altitude_ft * 0.3048;

```

```
% reading atmosphere data
data = readtable('ATMTABLE.xlsx');

altitudes      = data.Altitude_m;           % altitude (m)
temperature_K  = data.Temperature_K;        % temp (K)
pressure_Pa    = data.Pressure_Pa;          % pressure (Pa)


% interpolation and extrapolation (if we need )
T_0 = interp1(altitudes, temperature_K, altitude_m, 'linear', 'extrap');
P_0 = interp1(altitudes, pressure_Pa,   altitude_m, 'linear', 'extrap');

%% calculations

% defined matrices
F_M_0 = zeros(length(Mach_array), length(pi_c_array));  % Specific Thrust (F/M_0)
S = zeros(length(Mach_array), length(pi_c_array));      % Specific Fuel Consumption (S)
f_n = zeros(length(Mach_array), length(pi_c_array));    % Total fuel-air ratio
eta_P = zeros(length(Mach_array), length(pi_c_array));  % Propulsive efficiency
eta_T = zeros(length(Mach_array), length(pi_c_array));  % Thermal efficiency
eta_O = zeros(length(Mach_array), length(pi_c_array));  % Overall efficiency

%% loop

for i = 1:length(Mach_array)
    M_0 = Mach_array(i);
    V_0 = M_0 * sqrt(gamma * R * T_0);


        for j = 1:length(pi_c_array)
            pi_c = pi_c_array(j);
```

```
            %PCA analysis
            R = ((gamma-1)/gamma)*c_p ;
            a_0 = sqrt(gamma * R * g_c * T_0) ;
            tau_r = 1 + ((gamma-1)/2)* M_0 ^ 2 ;
            tau_lambda = T_t4 / T_0 ;
            tau_c = (pi_c)^((gamma-1)/gamma);
            tau_t = 1 - (tau_r *(tau_c-1))/tau_lambda;
            tau_lambdaAB = T_t7/T_0;

            % fuel/air ratio
            % this f formula depends on the pi_c:
            f = ((c_p * T_0) / h_PR) * (tau_lambda - tau_r * tau_c );

            % V_exit
            V_9 = a_0 * sqrt( (2/(gamma-1)) * tau_lambdaAB * (1 - 1/(
                tau_r * tau_c * tau_t)) );

            % specific thrust (F/dot{m}_0)
            F_M_0(i,j) = (a_0 / gc) * ((V_9 / a_0) - M_0);

            % Specific fuel consumption (S)
            S(i,j) = f * 1e6 / (F_M_0(i,j));

            % f
            f_n(i,j) = f;
            ftotal(i,j) = (c_p*T_0 / h_PR)*(tau_lambdaAB - tau_r);

            % efficiencies
            eta_P(i,j) = (2 * M_0) / ((V_9 / a_0) + M_0);
            eta_T(i,j) = ((gamma - 1) * c_p * T_0 * ((V_9 / a_0)^2 -
                M_0^2)) / (2 * ftotal(i,j) * h_PR);
            eta_O(i,j) = eta_P(i,j) * eta_T(i,j);


        end
end
```

```
%% graphss

figure % F_m0 vs pi_c
hold on
colors = lines(5);
legend_labels = {'M=0.5 ', 'M=1', 'M=1.5', 'M=2', 'M=2.5'};
h = gobjects(5,1);
for i = 1:5
    h(i) = plot(pi_c_array, F_M_0(i,:), '-', 'Color', colors(i,:),
        LineWidth= 2);
end
title('F_{m0} vs \pi_{c}');
xlabel('\pi_{c}');
ylabel('F_{m0}');
grid off
legend(h, legend_labels);
hold off


figure % S vs pi_c
hold on
colors = lines(5);
legend_labels = {'M=0.5 ', 'M=1', 'M=1.5', 'M=2', 'M=2.5'};
h = gobjects(5,1);
for i = 1:5
    h(i) = plot(pi_c_array, S(i,:), '-', 'Color', colors(i,:),
        LineWidth= 2);
end
title('S vs \pi_{c}');
xlabel('\pi_{c}');
ylabel('S');
grid off
legend(h, legend_labels);
hold off

figure %% f vs pi_c
hold on
colors = lines(5);
legend_labels = {'M=0.5', 'M=1', 'M=1.5', 'M=2', 'M=2.5'};
```

```
h = gobjects(5,1);

for i = 1:5
    h(i) = plot(pi_c_array, f_n(i,:), '-', 'Color', colors(i,:), '
        LineWidth', 2);
end

title('f vs \pi_{c}');
xlabel('\pi_{c}');
ylabel('f');
grid on
legend(h, legend_labels);
hold off

figure %% etaP vs pi_c
hold on
colors = lines(5);
legend_labels = {'M=0.5', 'M=1', 'M=1.5', 'M=2', 'M=2.5'};
h = gobjects(5,1);

for i = 1:5
    h(i) = plot(pi_c_array, eta_P(i,:), '-', 'Color', colors(i,:), '
        LineWidth', 2);
end

title('\eta_{p} vs \pi_{c}');
xlabel('\pi_{c}');
ylabel('\eta_{p}');
grid on
legend(h, legend_labels);
hold off

figure %% etaT vs pi_c
hold on
colors = lines(5);
legend_labels = {'M=0.5', 'M=1', 'M=1.5', 'M=2', 'M=2.5'};
h = gobjects(5,1);

for i = 1:5
```

```
    h(i) = plot(pi_c_array, eta_T(i,:), '-', 'Color', colors(i,:), 'LineWidth', 2);
end

title('\eta_{T} vs \pi_{c}');
xlabel('\pi_{c}');
ylabel('\eta_{T}');
grid on
legend(h, legend_labels);
hold off

figure %% eta0 vs pi_c
hold on
colors = lines(5);
legend_labels = {'M=0.5', 'M=1', 'M=1.5', 'M=2', 'M=2.5'};
h = gobjects(5,1);

for i = 1:5
    h(i) = plot(pi_c_array, eta_0(i,:), '-', 'Color', colors(i,:), 'LineWidth', 2);
end

title('\eta_{0} vs \pi_{c}');
xlabel('\pi_{c}');
ylabel('\eta_{0}');
grid on
legend(h, legend_labels);
hold off


pi_c_array = [1 5 10 15 20];  % we defined the pi_c for testing.
pi_c_plot = [1 5 10 15 20];

figure;
hold on;
colors = lines(length(pi_c_plot));
legend_labels = strings(1, length(pi_c_plot));
h = gobjects(length(pi_c_plot),1);

```

```
for k = 1:length(pi_c_plot)
    j = find(pi_c_array == pi_c_plot(k));
    h(k) = plot(Mach_array, f_n(:, j), '-', 'Color', colors(k,:), '
        LineWidth', 2);
    legend_labels(k) = sprintf('\\pi_c = %d', pi_c_plot(k));
end

title('f vs M_0');
xlabel('M_0');
ylabel('f');
legend(h, legend_labels, 'Location', 'best');
grid on;
hold off;

elseif strcmp(ab_status, "off")
    disp("Afterburner is OFF. PCA analysis is starting...");

    %inputs
M_0_start = input("mach start value:");
M_0_end = input("mach end value:");
altitude_ft = input("altitude:");
T_t4 = input("T_t4:");
%T_t7 = input("T_t7:");
pi_c_start = input("pi_c start value:");
pi_c_end = input("pi_c end value:");
% Mach and pi_c arrays
Mach_array = M_0_start : 0.5 : M_0_end;
pi_c_array = pi_c_start : 5 : pi_c_end;
gc=1;
% constants
gamma = 1.4;                    % specific heats ratio
c_p = 1004.83;                   % cp (J/kg.K)
h_PR = 42800e3;                  % hpr (J/kg)
R = ((gamma-1)/gamma)*c_p ;    % universal gas constant (J/kg.K)
g_c = 1; %its 1 since we work in SI units

%% important variables
% converting altitude to meter from feet
altitude_m = altitude_ft * 0.3048;
```

```

% reading the data table
data = readtable('ATMTABLE.xlsx');

altitudes     = data.Altitude_m;           % altitude (m)
temperature_K = data.Temperature_K;          % temperature(K)
pressure_Pa    = data.Pressure_Pa;          % pressure (Pa)

% interpolation and extrapolation (if we need)
T_0 = interp1(altitudes, temperature_K, altitude_m, 'linear', 'extrap');
P_0 = interp1(altitudes, pressure_Pa,   altitude_m, 'linear', 'extrap');
%% calculations

% Matrisces
F_M_0 = zeros(length(Mach_array), length(pi_c_array));   % Specific Thrust (F/M_0)
S = zeros(length(Mach_array), length(pi_c_array));      % Specific Fuel Consumption (S)
f_n = zeros(length(Mach_array), length(pi_c_array)); % Total fuel-air ratio
eta_P = zeros(length(Mach_array), length(pi_c_array));  % Propulsive efficiency
eta_T = zeros(length(Mach_array), length(pi_c_array));  % Thermal efficiency
eta_O = zeros(length(Mach_array), length(pi_c_array));  % Overall efficiency

%% calculation loop

for i = 1:length(Mach_array)
    M_0 = Mach_array(i);
    V_0 = M_0 * sqrt(gamma * R * T_0);


        for j = 1:length(pi_c_array)
            pi_c = pi_c_array(j);

```

```
            %PCA analysis
            R = ((gamma-1)/gamma)*c_p ;
            a_0 = sqrt(gamma * R * g_c * T_0) ;
            tau_r = 1 + ((gamma-1)/2)* M_0 ^ 2 ;
            tau_lambda = T_t4 / T_0 ;
            tau_c = (pi_c)^((gamma-1)/gamma);
            tau_t = 1 - (tau_r *(tau_c-1))/tau_lambda;


            % fuel/air ratio
            f = ((c_p * T_0) / h_PR) * (tau_lambda - tau_r * tau_c );

            % V_exit
            V_9 = a_0 * sqrt( (2 / (gamma - 1)) * (tau_lambda * tau_t -
                1) );
            Z = V_9/a_0; %V_9 /a_0


            % specific thrust (F/dot{m}_0)
            F_M_0(i,j) = (a_0 / gc) * ((V_9 / a_0) - M_0);


            % Specific fuel consumption (S)
            S(i,j) = f * 1e6 / (F_M_0(i,j));

            % efficiencies
            eta_P(i,j) = (2 * M_0) / ((V_9 / a_0) + M_0);
            eta_T(i,j) = (1-(1/(tau_c*tau_r)));
            eta_O(i,j) = eta_P(i,j) * eta_T(i,j);

            % the array of f
            f_n(i,j) = f;

        end
end


%% graphs

```

```
figure % F_m0 vs pi_c
hold on
colors = lines(5);
legend_labels = {'M=0.5 ', 'M=1', 'M=1.5', 'M=2', 'M=2.5'};
h = gobjects(5,1);
for i = 1:5
    h(i) = plot(pi_c_array, F_M_0(i,:), '-', 'Color', colors(i,:),
        LineWidth= 2);
end
title('F_{m0} vs \pi_{c}');
xlabel('\pi_{c}');
ylabel('F_{m0}');
grid off
legend(h, legend_labels);
hold off


figure % S vs pi_c
hold on
colors = lines(5);
legend_labels = {'M=0.5 ', 'M=1', 'M=1.5', 'M=2', 'M=2.5'};
h = gobjects(5,1);
for i = 1:5
    h(i) = plot(pi_c_array, S(i,:), '-', 'Color', colors(i,:),
        LineWidth= 2);
end
title('S vs \pi_{c}');
xlabel('\pi_{c}');
ylabel('S');
grid off
legend(h, legend_labels);
hold off



figure %% f vs pi_c
hold on
colors = lines(5);
legend_labels = {'M=0.5', 'M=1', 'M=1.5', 'M=2', 'M=2.5'};
```

```
h = gobjects(5,1);

for i = 1:5
    h(i) = plot(pi_c_array, f_n(i,:), '-', 'Color', colors(i,:), '
        LineWidth', 2);
end

title('f vs \pi_{c}');
xlabel('\pi_{c}');
ylabel('f');
grid on
legend(h, legend_labels);
hold off

figure %% etaP vs pi_c
hold on
colors = lines(5);
legend_labels = {'M=0.5', 'M=1', 'M=1.5', 'M=2', 'M=2.5'};
h = gobjects(5,1);

for i = 1:5
    h(i) = plot(pi_c_array, eta_P(i,:), '-', 'Color', colors(i,:), '
        LineWidth', 2);
end

title('\eta_{p} vs \pi_{c}');
xlabel('\pi_{c}');
ylabel('\eta_{p}');
grid on
legend(h, legend_labels);
hold off



figure %% etaT vs pi_c
hold on
colors = lines(5);
legend_labels = {'M=0.5', 'M=1', 'M=1.5', 'M=2', 'M=2.5'};
h = gobjects(5,1);
```

```
for i = 1:5
    h(i) = plot(pi_c_array, eta_T(i,:), '-', 'Color', colors(i,:), '
        LineWidth', 2);
end

title('\eta_{T} vs \pi_{c}');
xlabel('\pi_{c}');
ylabel('\eta_{T}');
grid on
legend(h, legend_labels);
hold off



figure %% eta0 vs pi_c
hold on
colors = lines(5);
legend_labels = {'M=0.5', 'M=1', 'M=1.5', 'M=2', 'M=2.5'};
h = gobjects(5,1);

for i = 1:5
    h(i) = plot(pi_c_array, eta_0(i,:), '-', 'Color', colors(i,:), '
        LineWidth', 2);
end

title('\eta_{0} vs \pi_{c}');
xlabel('\pi_{c}');
ylabel('\eta_{0}');
grid on
legend(h, legend_labels);
hold off

else
    error("wrong input please write 'on' or 'off'.");
end
```

6.2 MATLAB GUI CODE

```
function UnifiedTurbojetGUI
%% Main window
fig = uifigure('Name', 'Unified Turbojet GUI', 'Position', [100 100 1000 600]);

%% === Input Panel ===
inputPanel = uipanel(fig, 'Title', 'Input Parameters', 'Position', [20 310 300 260]);
inputPanel.BackgroundColor = [1 1 1 0.8];

uilabel(inputPanel, 'Text', 'Mach Range:', 'Position', [10 200 100 22]);
machField = uieditfield(inputPanel, 'text', 'Position', [120 200 150 22], 'Value', '0.5:0.5:2.5');

uilabel(inputPanel, 'Text', 'Altitude (ft):', 'Position', [10 170 100 22]);
altField = uieditfield(inputPanel, 'numeric', 'Position', [120 170 150 22], 'Value', 40000);

uilabel(inputPanel, 'Text', 'Tt4 (K):', 'Position', [10 140 100 22]);
Tt4Field = uieditfield(inputPanel, 'numeric', 'Position', [120 140 150 22], 'Value', 1666.67);

uilabel(inputPanel, 'Text', 'Tt7 (K):', 'Position', [10 110 100 22]);
Tt7Field = uieditfield(inputPanel, 'numeric', 'Position', [120 110 150 22], 'Value', 2222.22);

uilabel(inputPanel, 'Text', '\pi_c Range:', 'Position', [10 80 100 22]);
piField = uieditfield(inputPanel, 'text', 'Position', [120 80 150 22], 'Value', '5:5:30');

% Afterburner switch
abSwitch = uiswitch(inputPanel, 'slider', ...
```

```
        'Items', {'OFF', 'ON'},'Position', [120 50 50 30],  'Value', 'ON');

    uilabel(inputPanel, 'Text', 'Afterburner:', 'Position', [10 50 100 22]);

    % Buttons
    runBtn = uibutton(inputPanel, 'Text', 'Run', 'Position', [40 10 100 30], 'ButtonPushedFcn', @(btn, event) calculate());
    resetBtn = uibutton(inputPanel, 'Text', 'Reset', 'Position', [160 10 100 30], 'ButtonPushedFcn', @(btn, event) resetFields());

    %% === Plot Panel ===
    ax = uiaxes(fig, 'Position', [350 100 620 450]);
    title(ax, 'Choose a graph');

    %% === Graph Selection Panel ===
    plotPanel = uipanel(fig, 'Title', 'Select Graph', 'Position', [20 100 300 200]);
    plotPanel.BackgroundColor = [1 1 1 0.8];

    graphButtons = {
        'Specific Thrust vs CPR', 'F';
        'Specific Fuel Consumption  vs CPR', 'S';
        'Fuel-air ratio vs CPR', 'f';
        'Fuel-air ratio vs Mach Number', 'fM0';
        'Propulsive - Thermal - Overall efficiency vs CPR', 'eta';
    };

    for i = 1:size(graphButtons,1)
        uibutton(plotPanel, 'Text', graphButtons{i,1}, ...
            'Position', [20 150 - (i-1)*30 260 25], ...
            'ButtonPushedFcn', @(btn, event) plotGraph(graphButtons{i,2}));
    end

    %% === Global Variables ===
    results = struct();

```

```
%% === Calculation Function ===
function calculate()
    Mach_array = eval(machField.Value);
    altitude_ft = altField.Value;
    T_t4 = Tt4Field.Value;
    T_t7 = Tt7Field.Value;
    pi_c_array = eval(piField.Value);
    ab_on = strcmp(abSwitch.Value, 'ON');

    altitude_m = altitude_ft * 0.3048;
    data = readtable('ATMTABLE.xlsx');
    T_0 = interp1(data.Altitude_m, data.Temperature_K, altitude_m,
        'linear', 'extrap');
    P_0 = interp1(data.Altitude_m, data.Pressure_Pa, altitude_m, '
        linear', 'extrap');

    gamma = 1.4;
    c_p = 1004.83;
    h_PR = 42800e3;
    R = ((gamma - 1) / gamma) * c_p;
    g_c = 1;

    F_M_0 = zeros(length(Mach_array), length(pi_c_array));
    S = F_M_0;
    f_n = F_M_0;
    eta_P = F_M_0; eta_T = F_M_0; eta_O = F_M_0;


   for i = 1:length(Mach_array)
        M_0 = Mach_array(i);
        for j = 1:length(pi_c_array)
            pi_c = pi_c_array(j);

            a_0 = sqrt(gamma * R * g_c * T_0);
            tau_r = 1 + ((gamma - 1)/2) * M_0^2;
            tau_c = pi_c^((gamma - 1)/gamma);

            if ab_on
                tau_lambda = T_t4 / T_0;
```

```
                    tau_t = 1 - (tau_r * (tau_c - 1)) / tau_lambda;
                    tau_lambdaAB = T_t7 / T_0;
                    f = ((c_p * T_0) / h_PR) * (tau_lambdaAB - tau_r *
                        tau_c);
                    ftotal(i,j) = (c_p*T_0 / h_PR)*(tau_lambdaAB -
                        tau_r);
                    V_9 = a_0 * sqrt((2 / (gamma - 1)) * tau_lambdaAB *
                         (1 - 1/(tau_r * tau_c * tau_t)));
                    eta_P(i,j) = (2 * M_0) / ((V_9 / a_0) + M_0);
                    eta_T(i,j) = ((gamma - 1) * c_p * T_0 * ((V_9 / a_0
                        )^2 - M_0^2)) / (2 * ftotal(i,j) * h_PR);
                    eta_O(i,j) = eta_P(i,j) * eta_T(i,j);


                else
                    tau_lambda = T_t4 / T_0;
                    tau_t = 1 - (tau_r * (tau_c - 1)) / tau_lambda;
                    f = ((c_p * T_0) / h_PR) * (tau_lambda - tau_r *
                        tau_c);
                    V_9 = a_0 * sqrt((2 / (gamma - 1)) * (tau_lambda *
                        tau_t - 1));
                    eta_P(i,j) = (2 * M_0) / ((V_9 / a_0) + M_0);
                    eta_T(i,j) = (1-(1/(tau_c*tau_r)));
                    eta_O(i,j) = eta_P(i,j) * eta_T(i,j);

                end

                F_M_0(i,j) = (a_0 / g_c) * ((V_9 / a_0) - M_0);
                S(i,j) = f * 1e6 / F_M_0(i,j);
                f_n(i,j) = f;
            end
        end
        results.Mach = Mach_array;
        results.pi_c = pi_c_array;
        results.F_M_0 = F_M_0;
        results.S = S;
        results.f = f_n;
        results.etaP = eta_P;
        results.etaT = eta_T;
```

```
        results.eta0 = eta_0;

        title(ax, 'Choose a graph');
        cla(ax);
    end

%% === Plotting Function ===
function plotGraph(type)
    if isempty(fieldnames(results))
        uialert(fig, 'Run the calculation first.', 'Warning');
        return;
    end

    switch type
        case 'F'
            plot(ax, results.pi_c, results.F_M_0, 'LineWidth', 2);
            xlabel(ax, '$\pi_c$',                          'Interpreter','latex');
            ylabel(ax, '$\displaystyle \frac{F}{\dot m_0}$','Interpreter','latex');
            title (ax, '$\displaystyle \frac{F}{\dot m_0}\ vs\ \pi_c$', 'Interpreter','latex');
            legend(ax, compose('M=%.1f', results.Mach), 'Location', 'best');

        case 'S'
            plot(ax, results.pi_c, results.S, 'LineWidth', 2);
            xlabel(ax, '$\pi_c$',                 'Interpreter','latex');
            ylabel(ax, '$S$',                     'Interpreter','latex');
            title(ax, '$S\ vs\ \pi_c$',           'Interpreter','latex');
            legend(ax, compose('M=%.1f', results.Mach), 'Location', 'best');

        case 'f'
            plot(ax, results.pi_c, results.f, 'LineWidth', 2);
            xlabel(ax, '$\pi_c$',                 'Interpreter','latex');
            ylabel(ax, '$f$',                     'Interpreter','latex');
```

```
            title(ax, '$f\ vs\ \pi_c$',          'Interpreter','latex');
            legend(ax, compose('M=%.1f', results.Mach), 'Location', 'best');


        case 'fM0'
            plot(ax, results.Mach, results.f, 'LineWidth', 2);
            xlabel(ax, '$M_0$',                  'Interpreter','latex');
            ylabel(ax, '$f$',                     'Interpreter','latex');
            title(ax, '$f\ vs\ M_0$',            'Interpreter','latex');
            legend(ax, compose('\\pi_c = %.0f', results.pi_c), 'Location', 'best');

        case 'eta'
            cla(ax); hold(ax, 'on');
            for i = 1:length(results.Mach)
                M0 = results.Mach(i);
                % eta_P
                plot(ax, results.pi_c, results.etaP(i,:), '-', ...
                     'LineWidth', 1.5, 'DisplayName', sprintf('\\eta_P, M=%.1f', M0));
                % eta_T
                plot(ax, results.pi_c, results.etaT(i,:), '--', ...
                     'LineWidth', 1.5, 'DisplayName', sprintf('\\eta_T, M=%.1f', M0));
                % eta_0
                plot(ax, results.pi_c, results.eta0(i,:), ':', ...
                     'LineWidth', 1.5, 'DisplayName', sprintf('\\eta_0, M=%.1f', M0));
            end
            hold(ax, 'off');

            xlabel(ax, '$\pi_c$',                                    'Interpreter','latex');
            ylabel(ax, '$\eta$',                                     'Interpreter','latex');
```

```
            title(ax, 'Efficiencies vs $\pi_c$ for each Mach',
                        'Interpreter','latex');
            legend(ax, 'Location', 'best', 'Interpreter', 'tex');
    end
end


    %% === Reset Function ===
    function resetFields()
        machField.Value = '0.5:0.5:2.5';
        piField.Value = '5:5:30';
        altField.Value = 40000;
        Tt4Field.Value = 1666.67;
        Tt7Field.Value = 2222.22;
        abSwitch.Value = 'OFF';
        title(ax, 'Choose a graph');
        cla(ax);
        results = struct();
    end
end
```